Name – Deepanshu
Roll no – 2401010133
Section – C
Subject – ADA LAB
ASSIGNMENT NO – 3
Link - https://github.com/001-Deepanshu/ADA-LAB-ASSIGNMENT-3

hello.py > bfs

```python
import heapq
from collections import deque
import sys

graph = {
    0: [(1, 4), (2, 1)],
    1: [(0, 4), (2, 2), (3, 1)],
    2: [(0, 1), (1, 2), (3, 5)],
    3: [(1, 1), (2, 5)]
}

print("Graph (Adjacency List):", graph)

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    sys.stdout.write("BFS: ")
    while queue:
        node = queue.popleft()
        if node not in visited:
            sys.stdout.write(str(node) + " ")
            visited.add(node)
            for neighbor, _ in graph[node]:
                queue.append(neighbor)
    print()

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    sys.stdout.write("DFS: ")
    while stack:
        node = stack.pop()
        if node not in visited:
            sys.stdout.write(str(node) + " ")
            visited.add(node)
            for neighbor, _ in reversed(graph[node]):
                stack.append(neighbor)
    print()

bfs(graph, 0)
dfs_iterative(graph, 0)

def topo_sort(graph):
```

hello.py > bfs

```python
def topo_sort(graph):
    visited = set()
    rec_stack = set()
    stack = []

    def dfs_util(node):
        if node in rec_stack:
            raise Exception("Cycle detected! Not a DAG")

        if node not in visited:
            rec_stack.add(node)
            for neighbor, _ in graph[node]:
                dfs_util(neighbor)
            rec_stack.remove(node)
            visited.add(node)
            stack.append(node)

    for node in graph:
        if node not in visited:
            dfs_util(node)

    return stack[::-1]

try:
    print("Topological Sort:", topo_sort(graph))
except Exception as e:
    print("Topological Sort Error:", e)

def dijkstra(graph, start):
    for u in graph:
        for v, w in graph[u]:
            if w < 0:
                raise ValueError("Dijkstra cannot handle negative weights")

    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        current_dist, node = heapq.heappop(pq)

        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
```

```python
                heapq.heappush(pq, (new_dist, neighbor))

    return dist

def bellman_ford(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0

    edges = []
    for u in graph:
        for v, w in graph[u]:
            edges.append((u, v, w))

    for _ in range(len(graph) - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print("Negative cycle detected!")
            return None

    return dist

print("Dijkstra:", dijkstra(graph, 0))
print("Bellman-Ford:", bellman_ford(graph, 0))

def prim(graph):
    start = list(graph.keys())[0]
    visited = set([start])
    edges = []

    for to, weight in graph[start]:
        heapq.heappush(edges, (weight, start, to))

    mst = []
    total_cost = 0

    while edges:
        weight, frm, to = heapq.heappop(edges)

        if to not in visited:
            visited.add(to)
```

```python
            mst.append((frm, to, weight))
            total_cost += weight

            for next_node, w in graph[to]:
                if next_node not in visited:
                    heapq.heappush(edges, (w, to, next_node))

    if len(visited) != len(graph):
        print("Graph is disconnected!")

    return mst, total_cost


def kruskal(graph):
    parent = {}

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        parent[find(x)] = find(y)

    edges = set()
    for u in graph:
        parent[u] = u
        for v, w in graph[u]:
            if (v, u) not in edges:
                edges.add((u, v, w))

    edges = [(w, u, v) for (u, v, w) in edges]
    edges.sort()

    mst = []
    total_cost = 0

    for w, u, v in edges:
        if find(u) != find(v):
            union(u, v)
            mst.append((u, v, w))
            total_cost += w

    return mst, total_cost


print("Prim's MST:", prim(graph))
print("Kruskal's MST:", kruskal(graph))
```

Running code

```
Graph (Adjacency List): {0: [(1, 4), (2, 1)], 1: [(0, 4), (2, 2), (3, 1)], 2: [(0, 1), (1, 2), (3, 5)], 3: [(1, 1), (2, 5)]}
BFS: 0 1 2 3 38 |
DFS: 0 1 2 3 53 |
Topological Sort Error: Cycle detected! Not a DAG
Dijkstra: {0: 0, 1: 3, 2: 1, 3: 4}
Bellman-Ford: {0: 0, 1: 3, 2: 1, 3: 4}
Prim's MST: ([(0, 2, 1), (2, 1, 2), (1, 3, 1)], 4)
Kruskal's MST: ([(0, 2, 1), (1, 3, 1), (1, 2, 2)], 4)
```